project2_py\plotters.py

```python
import numpy as np
import matplotlib.pyplot as plt
try:
    from .project2 import optimize_with_history
except ImportError:
    from project2 import optimize_with_history

def plot_contour(a, b, z, path=None, ax=None, draw_contour = True):
    # Get range of function values
    Z_min = np.min(z)
    Z_max = np.max(z)

    # Define the number of contour lines & power factor
    num_levels = 100
    power_factor = 3

    positive_levels = Z_max * (np.linspace(0, 1, num_levels) ** power_factor)

    # If there are values below zero, plot them
    negative_levels = []
    if Z_min < 0:
        # Sample values below 0 for half of num_levels contours
        negative_levels = abs(Z_min) * (np.linspace(0, 1, num_levels // 2) ** power_factor)
        negative_levels = -np.sort(negative_levels)[::-1] # Sort in ascending order and flip to negative

    all_levels = np.unique(np.concatenate((negative_levels, positive_levels)))

    if ax is None:
        ax = plt.gca()
        show_plot = True
    else:
        show_plot = False

    if draw_contour: ax.contour(a, b, z, levels=all_levels, colors="#282853", linestyles='solid', linewidths=0.5)

    # If there's an optimization path, plot it
    if path is not None:
        path = np.array(path)
        if draw_contour == True:
            ax.plot(path[:, 0], path[:, 1], 'r', linewidth = 0.75, label="Optimization Path")
            ax.plot(path[0, 0], path[0, 1], 'go', label = "Start")
            ax.plot(path[-1, 0], path[-1, 1], 'm*', markersize=10, label="Finish")
        else:
            ax.plot(path[:, 0], path[:, 1], 'r', linewidth = 0.75)
            ax.plot(path[0, 0], path[0, 1], 'go')
            ax.plot(path[-1, 0], path[-1, 1], 'm*', markersize=10)

    from matplotlib.lines import Line2D
    custom_legend = [Line2D([0], [0], color="#282853", linewidth=0.5, label="Contour Lines")]
    if path is not None:
```

```
        custom_legend.extend([
            Line2D([0], [0], color="r", linewidth=0.75, label="Optimization Path"),
            Line2D([0], [0], color="g", marker="o", linestyle="None", label="Start"),
            Line2D([0], [0], color="m", marker="*", markersize=10, linestyle="None", label="Finish")
        ])
    ax.legend(handles=custom_legend, fontsize='x-small')
    ax.set_title("Contour Plot of Simple1 using QPM + ADAM")
    ax.set_xlabel("A")
    ax.set_ylabel("B")

    if show_plot:
        plt.show()


def plot_convergence(problem):
    plt.figure()
    for _ in range(3):
        problem._reset()
        path = optimize_with_history(
            problem.f,
            problem.g,
            problem.c,
            problem.x0(),
            problem.n,
            problem.count,
            problem.prob
        )
        iters = range(len(path))
        f_vals = np.zeros(len(path))
        for idx, value in enumerate(path):
            f_vals[idx] = problem.f(value)
        plt.plot(iters, f_vals)
    titles = [f'Run {i+1}' for i in range(3)] # More generic labels
    plt.xlabel("Iterations")
    plt.ylabel("Objective Value")
    plt.title(f"Objective Value vs # Iterations for {problem.prob} using QPM")
    plt.legend(titles)
    plt.grid(True)
    plt.show()


def plot_constraint(problem):
    plt.figure()
    for _ in range(3):
        problem._reset()
        path = optimize_with_history(
            problem.f,
            problem.g,
            problem.c,
            problem.x0(),
            problem.n,
            problem.count,
            problem.prob
        )
```

```
        iters = range(len(path))
        c_vals = np.zeros(len(path))
        for idx, value in enumerate(path):
            c_vals[idx] = np.max(problem.c(value))
        plt.plot(iters, c_vals)
    titles = [f'Run {i+1}' for i in range(3)] # More generic labels
    plt.xlabel("Iterations")
    plt.ylabel("Maximum Constraint Value")
    plt.title(f"Max Constraint vs # Iterations for {problem.prob} using QPM")
    plt.legend(titles)
    plt.grid(True)
    plt.show()



def plot_problem(problem, plot_size=3):
    """
    Visualizes 2D functions with optimization paths overlaid on contours.
    """
    def get_wrappedf(x1_val, x2_val):
        x = np.zeros(problem.xdim)
        if problem.xdim > 0: x[0] = x1_val
        if problem.xdim > 1: x[1] = x2_val
        return problem._wrapped_f(x)

    def get_max_constraint(x1_val, x2_val):
        x = np.zeros(problem.xdim)
        if problem.xdim > 0: x[0] = x1_val
        if problem.xdim > 1: x[1] = x2_val
        return np.max(problem.c(x))

    x1 = np.linspace(-plot_size, plot_size, 100)
    x2 = np.linspace(-plot_size, plot_size, 100)
    A, B = np.meshgrid(x1, x2)

    vectorized_f = np.vectorize(get_wrappedf)
    Z = vectorized_f(A, B)

    vectorized_c = np.vectorize(get_max_constraint)
    C_max = vectorized_c(A, B)

    _, ax = plt.subplots()
    plot_contour(A, B, Z, ax=ax, draw_contour=True)
    ax.contourf(A, B, C_max, levels=[-np.inf, 0], colors=["#E1FF00"], alpha=0.5)
    ax.contour(A, B, C_max, levels=[0], colors=['black'], linewidths = 1.5)

    path = None
    for _ in range(3):
        problem._reset()
        path = optimize_with_history(
            problem.f,
            problem.g,
            problem.c,
            problem.x0(),
```

```
            problem.n,
            problem.count,
            problem.prob
        )
        print(f"Number of Steps Taken: {len(path)}")
        print(f"Total Count: {problem.count()}")
        plot_contour(A, B, Z, path=path, ax=ax, draw_contour=False)

    print(f"Final constraints: {problem.c(path[-1])}")

    plt.tight_layout()
    plt.show()

```